Evaluation of the Multicore Performance Capabilities of the Next Generation Flight Computers

Marc Solé*† Jannis Wolf† Ivan Rodriguez*† Alvaro Jover†
Matina Maria Trompouki*† Leonidas Kosmidis†* David Steenari‡

*Universitat Politècnica de Catalunya (UPC) †Barcelona Supercomputing Center (BSC) ‡European Space Agency (ESA)

2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC) | 979-8-3503-3357-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/DASC58513.2023.10311151

***Abstract*—Multicore architectures are currently under adoption in the aerospace domain, however their software remains single-threaded. In this paper we argue about the benefits offered by homogeneous parallel processing, both in terms of performance, which is necessary for the implementation of advanced functionalities, as well as in terms of certification and in particular about mastering multicore interference.**

We discuss the implementation details of this programming paradigm in avionics and space real-time operating systems. We experimentally evaluate the performance benefits offered by several high-performance multicore platforms which are considered good candidates for next generation flight computers, using homogeneous parallel processing under a qualifiable real-time operating system used in the aerospace domain. Our results indicate near to linear speed-ups compared to traditional sequential processing, showing the benefits of this approach.

***Index Terms*—Multicore, performance, interference, homogeneous parallel processing, OpenMP, RTOS, RTEMS-SMP**

I. Introduction

Although the avionics and aerospace industries have recently started migrating towards multicore hardware architectures, at software level they currently use only single-threaded applications. While multicore platforms offer higher processing capacity than single core ones, i.e. a 4-core processor can execute 4 different applications/tasks concurrently, the actual aggregate performance is less than 4× due to resource sharing (i.e. shared caches, bus, memory) and intertask interference. Prior research works [1] [2] have shown that the single-threaded performance of software running on 4-core CPUs from different manufacturers can experience up to 9× slowdown and 12× respectively. This not only complicates the certification of airborne multicore systems according to AMC 20-193 [3] (formerly CAST-32A) [4], but also cannot satisfy the performance required for advanced on-board processing.

In particular, single core performance is not sufficient for the implementation of systems with an increased level of autonomy relying on computationally expensive algorithms such as Machine Learning (ML) and vision-based processing. Therefore, there is a need for parallel processing in airborne computers.

In this paper, we argue that *homogeneous parallel processing*, in which a demanding computation is spread among identical threads executing in a synchronised way on all available cores, can provide significant benefits, both at certification level as well as in terms of the obtained performance.

We discuss how homogeneous parallel processing can reduce the effort required to achieve compliance to AMC 20-193. Considering that avoiding the use of shared resources or mitigating their use is nearly impossible in multiprocessor platforms, homogeneous parallel processing limits the Worst Case Execution Time (WCET) analysis only to the interference among multiple instances of the same task, instead of any possible combination of tasks that can execute at the same time [5] [6] or pessimistic upperbounds of the single core execution time (i.e. 9× [1]).

We discuss the implementation details of this paradigm in the real-time operating systems used in the aerospace domain. In addition, in order to demonstrate the performance benefits of this approach, we provide experimental evidence of executing representative aerospace algorithms [7] on several multicore architectures which are considered for use in next generation flight computers. Our evaluation uses OpenMP and the space-qualified RTOS RTEMS-SMP, which is widely used in the aerospace domain. Our results compare the parallel performance of the selected platforms showing near linear speed-ups compared to their single-threaded performance, which can enable future advanced software capabilities.

II. Multicore Adoption in Aerospace

A. Multicore Processors for Aerospace Systems

Similar to all safety critical domains, the avionics and space sectors are moving towards multicore architectures [8], both for performance reasons, as well as due to the obsolescence of high performance single core processors in the market.

In the space domain, several processors particularly targeting this sector are available. Frontgrade Gaisler's Next Generation Multiprocessor (NGMP), based on a quad-core fault-tolerant LEON4 processor, has been available for over a decade, with its GR740 flight model being QML-Q and QML-V qualified in 2021 [9]. Currently, the octa-core GR765, based on both the SPARC-based LEON5 and RISC-V based NOEL-V cores is under development by Frontgrade Gaisler. Both de-

velopments have been funded by the European Space Agency (ESA). The European Union has funded the development of the quad-core DAHLIA processor [10], based on the ARM Cortex-R52, which is currently adopted as a hard processor in the NG-Ultra FPGA [11]. An open source multicore platform based on NOEL-V, extended with short vector capabilities and an open source RISC-V Graphics Processing Unit (GPU) prototype on an FPGA is under development [12] in the ongoing European Commission's METASAT project [13].

In the US, the 4 core RAD5545 is available from BAE Systems [14] [15]. NASA is also funding the development of Next-generation High Performance Spaceflight Computing (HPSC) processors. Boeing has been awarded a contract for the development of an ARM-based multicore system [16], with 8 ARM Cortex-A53 and dual core Cortex-R52 cores. Under the same program, Si-Five and Microchip were recently awarded a contract for the development of a multicore RISC-V based processor for space [17].

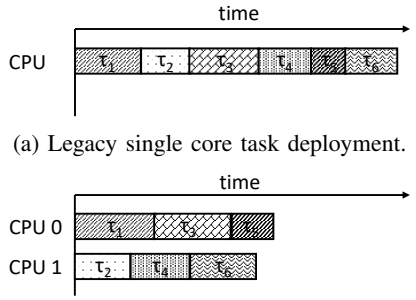
In addition to these radiation-hardened processors specifically designed for space, radiation-tolerant and Commercial Off-The-Shelf (COTS) multicore processors are considered in lower cost missions for New Space [18]. The Xilinx (now AMD) Zynq Ultrascale+ SoC (ZCU102) is an attractive platform combining a high performance multicore, with real-time processors and an FPGA, which is a common choice for space missions [19]. Multicore SoC with embedded GPUs, especially the ones developed for the automotive domain are also good candidates for space [18], including the NVIDIA TX2 and Xavier, as well as AMD V1605B.

In avionics, both PowerPC and ARM-based solutions are explored. The 8-core PowerPC NXP QorIQ P4080 [20] [21] and T2080 [22] are considered, as well as the 12-core NXP T4240 [23]. However, there is a preference towards the T series of NXP processors over the P ones, due to their increased performance and architectural improvements [24]. From the ARM-based multicore CPUs, NXP's LS1088 which features 8 ARM Cortex A53 CPUs is considered by industry [25].

B. Multicore Support in Real-Time Operating Systems

With the availability of multicore hardware, real-time operating systems (RTOSes) started including support for exploiting its capabilities. In the avionics domain, the implementation of Integrated Modular Avionics systems relies on an ARINC-653 [26] compliant Real-Time Operating System. The update of the ARINC-653 Part 1 Supplement 3, Required Services as well as the later published Part 1 Supplement 4 added support in the specification for the support of multicores. All ARINC-653 RTOS vendors are currently supporting multicores such as WindRiver VxWorks [24], DDC-I Deos [25], SYSGO PikeOS [20] and fentISS LithOS [27] to name a few.

While ARINC-653 RTOSes are also used in space, particularly for their avionics part, other RTOSes are also widely used. In particular, the open source RTEMS is a common choice used in numerous missions over the decades. Since version 5, RTEMS has obtained multicore support, known as RTEMS-SMP. ESA has recently performed the pre-qualification of the space-profile subset of RTEMS-SMP for the GR740 and GR712RC, and provides openly its pre-qualification data package [28].



(a) Legacy single core task deployment.

(b) Multicore task deployment in multicores used in aerospace.

Fig. 1: Current multicore migration concept in the aerospace domain. Single-threaded tasks that were executed on the same CPU in a single core CPU are now distributed in multiple CPUs. Note the increase in the execution time of each task due to multicore interference.

C. Software Deployment in Multicores

The shift from single core architectures to multicore ones in avionics is a disruptive move comparable to the one that the avionics sector experienced when migrating from federated to integrated architectures [29]. For this reason, the avionics industry seeks solutions which are backwards compatible and incremental [25]. The IMA concept is based on space and time partitioning. In a single core environment, IMA allows the development of software partitions in isolation and their integration and deployment with other partitions in the final system, as if they were executing alone.

When ARINC-653 Part 1 Supplement 4 added support for multicores, included a similar requirement; software developed for an ARINC-653 RTOS compliant with Part 1 Supplement 3 to run on a single core processor, shall also run on a single core of a multicore platform under an RTOS following Part 1 Supplement 4 with the same behaviour. This allows legacy avionics software to be migrated to multicore systems while maintaining portability. For this reason, this is the most popular approach followed currently by both industry [21] and academia [30]. In the space domain, in non-partitioned multicore operating systems such as RTEMS-SMP, the same approach is also currently adopted in multicore deployments.

Figure 1 depicts this concept. Single core tasks in RTEMS or partitions in ARINC-653 RTOSes running on a single core (Figure 1a), are distributed across the multiple cores of a multicore (Figure 1b). Theoretically, moving to an N-core multicore system, increases the capacity of the system by N times. However, in practice this is not true. Note that in Figure 1b, the execution time of each task/partition has been increased, due to *multicore interference*.

Multicore interference is introduced due to resource sharing between the different CPUs. This includes shared caches, i.e. the L2 cache or higher level caches, as well as DRAM.

Determining the execution time increase of each task/partition is a complex procedure, since it depends on the particular characteristics of the tasks/partitions that are executing in parallel and therefore are sharing resources.

Two tasks which are using the same resource, eg. the L2 cache, if they are executing at the same time in different CPUs will evict each other's data from L2, causing them a large slowdown. However, if one of the two tasks fits in the private L1 cache of their core, the slowdown will be smaller.

In order to deal with the multicore contention problem, AMC 20-193 [3] requires the user to identify all the interference channels that exist in the target multicore platform and can affect the software behaviour, and apply mitigations for the effects of contention (objective `MCP_Resource_Usage_3`).

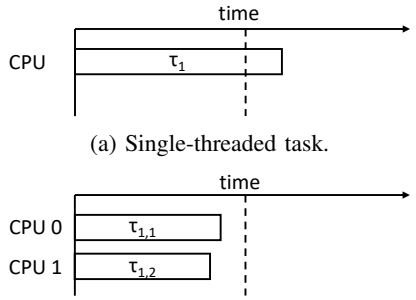
There are two ways of mitigating the effects of multicore interference. One approach is through robust partitioning. This allows to compute the Worst Case Execution Time (WCET) of each task in isolation (objective `MCP_Software_1`). This includes different cache partitioning methods. Cache partitioning can be performed either using way partitioning or cache colouring [31]. Another method for robust partitioning is to take into account the effect of multicore interference in the timing analysis, in an independent way from all different co-runners, using worst case contention generation software [25]. However, robust partitioning can be pessimistic and reduce the obtained performance. For example, (static) cache partitioning and cache colouring, reduce the available L2 cache space assigned to the software. If the working set of the software is larger than its L2 slice, its performance is degraded compared to its execution in isolation. For example, an application may experience up to a 2.5× slowdown by partitioning the L2 cache of a 4 core multicore [31].

The second approach to deal with resource interference is to take into account the different co-runners, and therefore the actual contention they generate. In this case, all combinations of co-running tasks are taken into account [5] [6]. However, the drawback of this approach is that the analysis needs to be repeated if the software or its schedule is changed.

III. Homogeneous Parallelism

While maintaining single core partitions and distributing them across the different cores facilitates the migration to multicore architectures, it is not enough for providing the required performance for the implementation of advanced functionalities. Even if the combined performance of a multicore is higher than a single core, a sequential task cannot take advantage of the multiple cores in the system.

For example, modern and future aerospace systems will require to perform an increasing amount of demanding computations, which cannot be satisfied with the single-threaded performance of modern CPUs. Figure 3 shows such an example, in which a task is not able to meet its performance target on a single core, using a sequential implementation (Figure 2a). However, if the computation is divided in identical tasks which perform the same operation but on different parts of the data, the performance target can be met. Note that due to multicore



(a) Single-threaded task.

(b) Parallel multithreaded task.

Fig. 2: Performance improvement of parallel tasks. The single thread implementation of a demanding algorithm cannot meet its performance target, but the parallel implementation can.

interference, the task execution time is not reduced to the half, but it is longer. Moreover, due to the non-determinism introduced by resource interference, the tasks do not finish exactly at the same time and neither with the same order.

We call this execution model *Homogeneous Parallelism*, because the tasks are executing the same code. Another work used the term Data Parallel decomposition [31]. Homogeneous parallelism is a very common parallel programming model, both in multicore CPUs as well as in other types of parallel processing. Variants of this programming model are known also as single instruction, multiple data (SIMD) as well as single instruction, multiple threads (SIMT) used in GPUs.

Apart from its performance benefits, homogeneous parallelism, when the parallel tasks are scheduled for execution in a synchronised way and in all available cores, has an additional benefit for certification. Since the co-runner tasks are fixed and identical, the generated contention has a fixed intensity and characteristics at any point of their execution. This means that each instance of the parallel task generates the same multicore contention and its execution time behaviour is affected in the same way. For this reason, there is no need for neither contention mitigation nor taking into account the effect in execution time received by running at the same time with other software.

This means, that the parallel task can be analysed in isolation, reducing significantly the multicore certification effort.

A. Implementation of Homogeneous Parallelism in Aerospace Systems

1) Implementation under ARINC-653 Part 1 Supplement 4: Until ARINC-653 Part 1 Supplement 3, which only supported single core partitions, the implementation of homogeneous parallelism under an ARINC-653 RTOS required splitting the parallel processing in identical partitions, scheduled for execution in the different cores of the system. This parallelisation scheme has been demonstrated in an aerospace case study by Thales Alenia Space [31], under the name Data Parallel decomposition. Such implementation requires that each of the identical partitions processes a different part of the input data and output data. For example, splitting the implementation in

two partitions requires that the first partition processes the first half of the data, and the second partition the second half.

ARINC-653 partitions offer both space and time partitioning. While the time partitioning in a multicore deployment facilitates the synchronised execution of the identical partitions on the different cores, which is required by the homogeneous parallelism programming model, space partitioning complicates the implementation of this task. Space partitioning means that one partition cannot access the memory of another partition, which is required for the implementation of the homogeneous parallelism pattern, i.e. accessing the input and the output data of the processing.

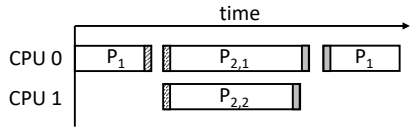
Figure 3a shows the complexity of this implementation. First, the processing that was once implemented in a single partition, now needs to be decomposed in at least 2 different types of partitions. The non-parallelisable part of the algorithm remains in partition P_1, which needs to share the data to be processed with the identical partitions $P_{2,1}$ and $P_{2,2}$, through an inter-partition communication mechanism. Depending on the access pattern of the particular parallelised algorithm, P_1 needs to share only an exclusive part of the data with each of the partitions, or the entire data with all partitions. The cost of the data communication depends on the actual inter-partition communication primitive used (e.g. queuing port) and its particular implementation by the RTOS, the size of the exchanged data and the number of partitions. Due to the ARINC-653 inter-partition communication semantics and space partitioning, the underlying implementation of inter-partition communication usually involves actual data copies instead of memory mapping, which increases their cost. In addition, the APEX inter-partition communication APIs require extra copies within the partitions in order to use the exchanged data. Moreover, since each partition operates in a distinct address space, the partitions are not able to take advantage of the presence of a shared cache to speed up access to shared data.

An additional drawback of the implementation of homogeneous parallelism under Part 1 Supplement 3 is the coupling between different partitions. This is against the basic IMA principle which enables the development and verification of each partition in isolation.

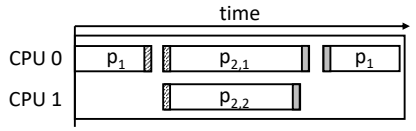
Therefore, despite its feasibility and benefits, the implementation of homogeneous parallelisation under ARINC-653 Part 1 Supplement 4 can be challenging.

2) Implementation under ARINC-653 Part 1 Supplement 5: The ARINC-653 Part 1 Supplement 5 published in 2019 introduced the notion of a multicore partition, i.e. a partition with processes running in multiple cores. Note that the notion of process in ARINC-653 is different than the POSIX one, and it is equivalent to POSIX threads. Therefore, ARINC-653 processes share the same address space within a partition.

This enables multithreaded processing within a partition. Figure 3b shows the implementation of homogeneous parallelism under this paradigm. Conceptually, the algorithm is decomposed in the same way as before. However, now all the processes belong in the same partition. Intra-partition



(a) Homogeneous parallelism implementation under ARINC-653 Part 1 Supplement 4. At least two different partitions are required, which exchange data through inter-partition communication mechanisms.



(b) Homogeneous parallelism implementation under ARINC-653 Part 1 Supplement 5. A multicore partition with processes running in different cores is used. The processes can exchange data through intra-partition communication mechanism, or directly access data, since they share the same address space.

Fig. 3: Implementation of homogeneous parallelism in different variants of an ARINC-653 RTOS.

scheduling and core mapping can be used to achieve the synchronised execution of the parallelised tasks.

A benefit compared to the previous approach is that since the processes share the same address space, intra-partition communication is significantly cheaper. Again the process p_1 which implements the sequential part of the software needs to pass the data to processes $p_{2,1}$ and $p_{2,2}$, which execute in parallel. This can be done either using intra-partition APEX communication primitives or by directly accessing the shared data if they are defined as global data within the partition.

Similar to the previous example, the inter-process communication cost depends on the actual primitive used (e.g. blackboard) and its particular implementation by the RTOS, the size of the exchanged data and the number of processes. Since the processes share the same address space, data transfers are faster and can take advantage of the shared cache.

If no intra-partition communication mechanism is used, but the data are accessed through shared global memory, the cost is minimised, as well as the implementation complexity. In this particular case, shared data accesses get a full performance benefit from the shared cache, as well as consistent data through the processor's cache coherence mechanism. If the parallel algorithm needs to write to a given memory position from multiple processes, atomic operations can be used.

While cache coherence is usually identified as a source of multicore interference [25] for multicore certification, note that in the cache of homogeneous parallelism this it does not present a problem that needs to be mitigated. As mentioned earlier, under this execution paradigm, all parallel threads (ARINC-653 processes) of the partition are executing exactly the same code and generate the same cache coherence traffic. Therefore, both the effects of cache coherence and shared cache contention are taken into account during their execution.

In contrast to the implementation of homogeneous parallelism under Part 1 Supplement 4, multicore partitions allow

```
1  #pragma omp parallel for
2  for (i = 0; i < n; i++) {
3    for (j = 0; j < n; j++) {
4      dot = 0;
5      for (k = 0; k < n; k++) {
6        dot += d_A[i*n+k]*B_transposed[j*n+k];
7      }
8      d_C[i*n+j ] = dot;
9    }
10 }
```

Fig. 4: Excerpt of transposed matrix multiplication parallelisation using OpenMP from the GPU4S Bench [7] [34].

```
dot = vmlaq_u8(dot, d_A_8x16, B_transposed_8x16);
```

(a) NEON intrinsic example

```
__usum(dot, d_A_8x4, "usmul", B_transposed_8x4);
```

(b) SPARROW intrinsic example.

Fig. 5: Examples of SIMD intrinsics implementing the dot product operation shown in line 6 of Figure 4, over 16 elements in NEON (a) and 4 elements in SPARROW [35] (b). The elements are 8-bit each and they are loaded from the `d_A` and `B_transposed` arrays.

the parallel implementation of an algorithm to remain within the boundaries of a single partition, facilitating in this way its development and validation completely in isolation.

3) Implementation under RTEMS: Unlike ARINC-653 RTOSes, RTEMS does not implement space partitioning, and therefore all tasks share the same flat address space. Therefore, homogeneous parallelism can be implemented similar to multicore partitions. RTEMS provides similar inter-task communication primitives which can be used if the data are chosen to be communicated between tasks, or the tasks can access directly the data if they defined as global data.

In addition to this manual implementation using programmer-defined tasks, RTEMS-SMP offers seamless support for homogeneous parallelism using the OpenMP programming model [32]. OpenMP is one of the oldest standardised parallel programming models, and the most widely used for shared memory parallel programming [33], especially in high performance computing. OpenMP supports multiple types of parallelism. For homogeneous parallel programming, which is the first one introduced in OpenMP and most widely used, it supports loop parallelisation.

This is shown in the code listing 4, with the parallelisation of a transposed matrix multiplication. Note that using OpenMP does not require any modification in the application source code. The source code is simply annotated using `#pragmas`, such as in line 1. If the code is compiled with a compiler which does not support OpenMP or does not have the compilation option for OpenMP parallelisation enabled, this line is ignored, and the code is compiled for sequential execution.

This line instructs the compiler to execute the `for` loop in parallel. The number of threads by default match the number of the cores in the system, as it is required by our homogeneous parallelism proposal, but it can also be configured. The threads can also be statically assigned to cores. When this line is encountered, OpenMP continues the execution of that parallel region using all threads executed in parallel. Similar to Figure 3, the OpenMP runtime takes care of passing the relevant input to the *forked* threads, and then collect their outputs when they are *joined*. For this reason, OpenMP is known as fork-join programming model.

In practice, the OpenMP threads are not dynamically forked or destroyed, especially under RTEMS, since this would create a significant overhead for its real-time execution. Instead, the threads are created at boot time, and then reused when a parallel region is executed. When all the threads of the parallel region finish their execution and reach a barrier, the data are passed back to the main thread of execution (p_1 in Figure 3).

Unlike the manual implementation of homogeneous parallelism, OpenMP does not require the data to be in a global scope. The relevant data are passed using pointers (which would be discouraged in a manual implementation) or copied if there is a need to do so, since the runtime can take the optimal decision. OpenMP offers also options for specifying atomic accesses to certain variables or memory positions, as well other information such as definition of reduction or private variables per thread. In general, OpenMP supports all features required to parallelise any sequentially written code.

In addition to homogeneous parallelism, OpenMP supports a task parallelism, in which various parts of a sequential code can be executed in parallel. However, since in the tasking subset the threads do not execute the same code, it does not offer the certification benefits we describe in this paper.

B. Vector/SIMD Processing

Vector instructions currently available in several processors are another form of homogeneous parallelism. Examples of this feature are the NEON instructions in ARM processors, SSE and AVX instructions in Intel processors, the Altivec instructions in PowerPC processors and the packed vector instructions and vector instructions of the RISC-V architecture.

Figure 6 shows the notion of SIMD performance improvement. A computationally intensive part of the sequential code can be accelerated using this type of instructions. These operations can be manually accessed at C code level using compiler intrinsics, i.e. function-like constructs that translate to a single processor instruction and applies the same operation in multiple data elements. The code listings of Figure 5 show two examples of processor intrinsics implementing the dot product operation in line 6 of the transposed matrix multiplication shown in Figure 4. In the case of ARM NEON, 16 multiply and accumulate operations of 8-bits are performed with a single SIMD instruction. In the case of the SPARROW AI Accelerator [36] which is used in the METASAT RISC-V based platform [37], the operation is applied over 4 elements.

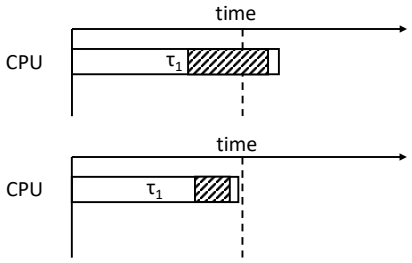


Fig. 6: Performance improvement using SIMD instructions to parallelise part of the sequential processing of a task.

Since this is a language feature, it is also available both in sequential code and in RTOSes, such as ARINC-653 and RTEMS, provided that the latter provides support for it. However, to our knowledge this feature is not used in avionics or space systems yet, although it is a very attractive solution to increase the guaranteed performance of critical systems [38].

SIMD instructions can be also combined with the homogeneous parallelism concept described in the previous subsections to achieve higher performance. This can be done by manually written code using intrinsics. Code with intrinsics can be annotated with pragmas, or code can be parallelised automatically by the compiler using the `#pragma SIMD` option of OpenMP. In the latter case, the compiler will try to use SIMD instructions for that particular part of the code.

Regardless of the homogeneous parallelism scheme used in combination with vector/SIMD instructions, the benefits remain the same. Since all threads execute exactly the same code, and their execution is synchronised and uses all cores in the system, the interference generated and received by each thread is identical. Therefore, there is no need for mitigations.

IV. Experimental Evaluation

In this section, we evaluate the performance benefits of homogeneous parallelism in several multicore processors which are considered for use in next generation flight computers. For the evaluation, we are using the open source GPU4S Bench [7], known also as OBPMark Kernels [34], part of the European Space Agency's OBPMark benchmarking suite [39].

The GPU4S Bench benchmarking suite contains parallelised implementations of several algorithmic building blocks which are commonly used in several applications domains in the aerospace sector, according to a survey performed in all software divisions of Airbus Defence and Space in the GPU4S ESA funded project [18]. Since we focus on multicore performance, we are using the OpenMP version of the GPU4S Benchmarks, which we have ported to RTEMS-SMP. Our port will be released in the official repository [34].

We are using the latest development version of RTEMS-SMP (RTEMS 6 at the time of writing) from the master branch of RTEMS, which includes support for ARM and RISC-V architectures. However, for GR740 we are using the RTEMS 5 version from Frontgrade Gaisler. We are evaluating a subset of the multicore platforms presented in Section II-A which are promising candidates for future flight computers.

Table I shows the details and configurations of our evaluated platforms. In the GR740 and METASAT platforms, RTEMS-SMP is executed natively. However, for the rest of the platforms we had difficulties to flash them and boot RTEMS-SMP. Since the rest of the platforms are based on Linux, we have done a series of modifications to allow near native, real-time performance such as using the real-time patches of the Linux Kernel and moving all interrupts and processes to the first core. Moreover, we are using the KVM virtualisation solution to execute RTEMS-SMP within a multicore virtual machine spanning all cores of the system. Note that although there is possibility for KVM to use cache partitioning [2] solutions, we have not used this option, since as we argue in our proposal, homogeneous parallelism does not require isolation.

A. Performance results

Figure 7 shows the performance of matrix multiplication for single precision floating point and 8-bit integer data types, for the platforms under analysis in terms of MFLOPS and MOPS respectively (number of floating point or integer operations per second). For ZCU102, TX2 and GR740 there is no performance difference between the two data types, but in the rest of the platforms int8 is faster: 2 times faster in the Xavier and 50% faster in the V1605B and 5 times in the METASAT platform. The difference is similar also for other integer types and depends on the number of integer and floating point units and their latency. For example, METASAT's NOEL-V core has 2 integer units but a single floating point unit (NanoFPUnv) which is not pipelined. Moreover, the performance seems to remain relatively constant regardless of the input size.

The same trend is observed in multicore execution. All platforms achieve near ideal speed-up (3.8-3.97× and 1.9 for Xavier's 10W power mode which uses 2 CPUs), except the TX2 which has lower scaling (2.1-2.5× and 3.1-3.5× depending on the power mode). This is an indication that homogeneous parallelism is effective and no cache interference mitigation is required. In fact, in our optimised implementation the second matrix is first transposed and then multiplied. This makes our implementation cache-friendly. Moreover, since in this parallelisation scheme each thread is computing the dot product of a different row, but the different threads are reusing the elements of the second matrix, they benefit from finding them in the shared cache.

Figure 8 presents the results of 8-bit integer matrix multiplication using SIMD instructions. In the case of ARM NEON, the speed-up of the Xavier over the sequential version is relatively independent of the input size and ranges from 2.3× in the 10W mode to 3.3-4× in the 15W mode. In the TX2 the speed-up starts from 9× for 1K×1K inputs, but gradually drops to 5× for 4K×4K. Despite that, NEON is faster than multicore parallelisation in this platform, as well as in Xavier in the 10W mode. In the METASAT platform, SPARROW instructions provide a 6× speed-up over the sequential version. When combined with OpenMP, SPARROW provides a 20× speed-up over the sequential and 5× over the multicore version. This is an 8 × improvement over the single core 8-

TABLE I: Evaluated Multicore Platforms and configurations used.

Platform	Node	CPUs	Frequency MHz	Shared Cache	Memory Size	Power
AMD ZCU102	16nm FinFET	4 ARM A53 and 2 ARM R5 Disabled	1500	2MB	4GB	22.8W
NVIDIA TX2	16nm TSMC	4 ARM A57 (enabled) & 2 Denver	1200	2MB	8 GB	7.5W
NVIDIA TX2	16nm TSMC	4 ARM A57 (enabled) & 2 Denver	2000	2MB	8 GB	7.5W
NVIDIA Xavier	12nm TSMC	4 ARM v8.2 Carmel (2 Enabled)	1200	L2 2MB (per pair) L3: 4MB	32 GB	10W
NVIDIA Xavier	12nm TSMC	4 ARM v8.2 Carmel (4 Enabled)	1200	L2 2MB (per pair) L3: 4MB	32 GB	15W
AMD V1605B	14nm GF	4 x86 Ryzen (hyperthreading disabled)	1600	2MB	16 GB	∼15W
Frontgrade Gaisler GR740	65nm ST	4 LEON4	250	2MB	256 MB	1.5W
METASAT	AMD VCU118 FPGA	4 NOEL-V + SPARROW	100	256KB	4GB	5.3W

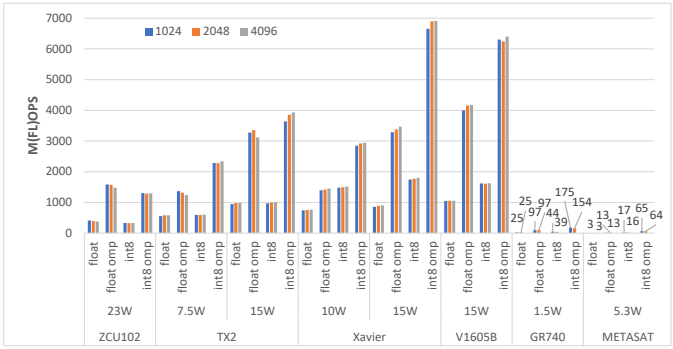
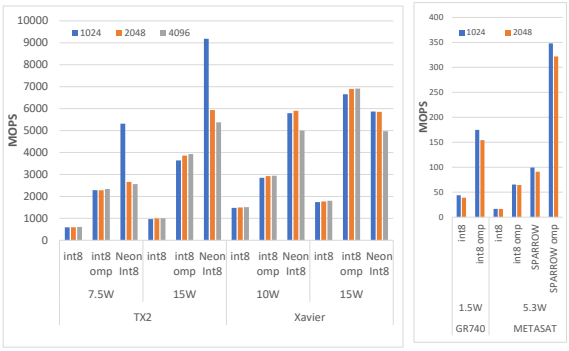


Fig. 7: Matrix Multiplication performance results.



(a) NEON (b) SPARROW

Fig. 8: Matrix multiplication results with SIMD instructions.

bit integer performance of GR740 and 2× over its multicore performance. This is very promising for the performance of the upcoming GR765 potentially coupled with SPARROW, considering that the METASAT platform is implemented in an FPGA, has a lower frequency, and uses lower performance components than the ones that GR765 will use, such as the non-pipelined floating point unit and the less performant L2C-Lite shared cache from the GPL GRLIB IP library.

Sliding FFT: Figure 9 shows the results of a sliding FFT applied for a window of 128 samples, used for processing ADS-B data. In this benchmark we notice that the performance increases with the input size increase for the most powerful platforms. The best performance is achieved by NVIDIA Xavier for the largest input size in the multicore configuration. Interestingly, the performance achieved by GR740 is very close to the one of the ZCU102, although its power consumption and its frequency are much lower. Moreover, as in the case of matrix multiplication very good speed-ups are observed. In the lower performance platforms, the speed-ups are above 3×, with the maximum scaling achieved by GR740 and METASAT, close to 3.9×. However, Xavier and V1605B have moderate multicore speed-ups between 1.6-2.6×.

FIR: Finite Impulse Response (FIR) filtering results are presented in Figure 11, for a filter with 64 taps. Again GR740's performance is in the same range with ZCU102 (2× difference). The highest performance is achieved by the V1605B both in single core as well as in multicore setting. The achieved speed-ups are lower for this benchmark. Again the Xavier and the V1605B have the smaller improvement over the sequential version, since they have processors with higher single-threaded performance. The speed-up of the ZCU102 ranges from 2.8-3.6×, while for the GR740 and METASAT it is steady over 3.5×.

2D Correlation: In terms of image correlation, which is shown in Figure 10, multicore performance provides small improvements, from 1.2-1.75×. The platform with the higher performance is NVIDIA Xavier at the 15W power mode. In this case also the ZCU102 and the GR740 are quite close, with a 2.5× difference.

2D Convolution: Figure 12 shows the results of image convolution with a 3×3 filter. In each of the platforms, the performance remains the same regardless of the increase in the input size and the multicore speed-up is consistently close to linear. NVIDIA Xavier in 15W is faster, followed by V1605B. GR740's performance is also similar to ZCU102, which is 50% faster.

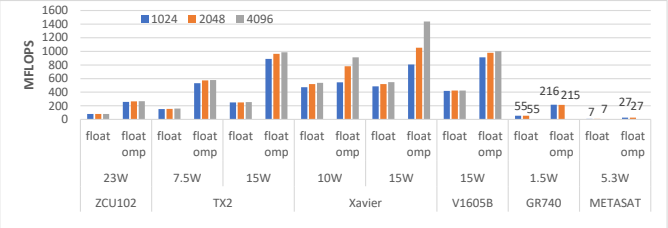


Fig. 9: Sliding FFT with a 128-sample window.

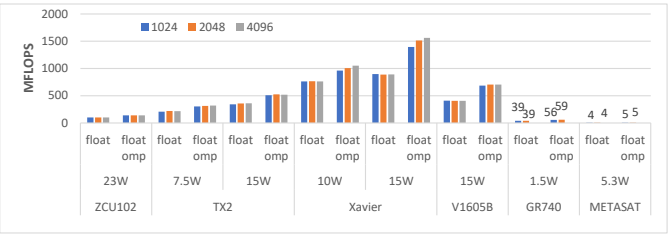


Fig. 10: 2D Correlation.

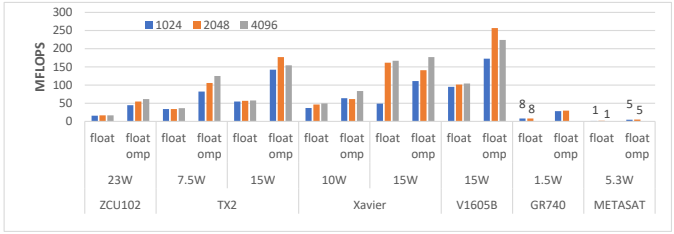


Fig. 11: Finite Impulse Response filtering with 64 taps.

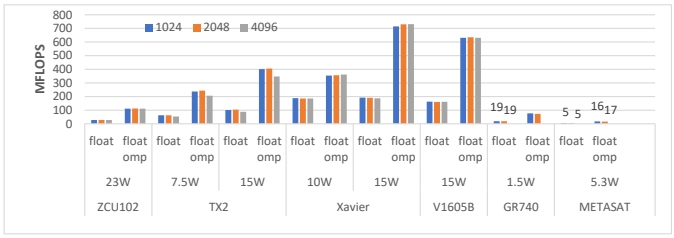


Fig. 12: 2D Convolution with a 3×3 kernel.

CIFAR-10 Inference: Figure 13 presents the results of an inference pipeline for the CIFAR-10 dataset. The pipeline consists of several different types of neural network layers such as convolutions, ReLU (rectifier linear unit), max pooling, local response normalisation, fully connected layers and softmax. The inference performance is measured in frames per second (FPS) and it remains steady regardless of the number of images. The AMD V1605B achieves the best performance in this task, followed by NVIDIA TX2 and Xavier, all at the 15W power mode. The ZCU102 is 2.2× faster than the GR740. All platforms provide speed-ups higher than 3×, however the NVIDIA Xavier shows a speed-up below 2×.

V. Related Work

Our proposal for homogeneous parallelism was first implemented by Thales Alenia Space in [31], under the name Data Parallel decomposition. However, in their implementation homogeneous parallelism did not use all available cores in the system, but instead two of them. For this reason, it required to take into account multicore contention with the rest of the partitions scheduled in the other cores. Moreover, their implementation was under Part 1 Supplement 4, using identical single core partitions, which require higher implementation effort and have higher overhead as discussed in Section III-A1.

Our proposal of homogeneous parallelism implemented on an RTOS compliant with ARINC-653 Part 1 Supplement 5 has some similarities with Melani et al. [30], which also allows the execution of multiprocessor partitions using all the cores of the system. However, their proposal of synchronous partition switches, focuses on the migration of legacy uniprocessor partitions to multicores. Our work differs on the fact that we only allow homogeneous parallelism within the partitions, so that we do not have to mitigate multiprocessor interference.

In terms of benchmarking the performance of aerospace processors, there are several similar works which have used parallel OpenMP and NEON implementations. Several works use the SHREC Space Bench benchmarking suite, developed by Tyler Lovelly at SHREC [40]. Similar to the GPU4S Bench [7] / OBPMark Kernels [34], it was based on a survey performed for common operations used in space processing [41]. Moreover, similar to GPU4S Bench it supports different data types (8, 16 and 32 bit integers and single and double floating point) and several programming models (sequential, OpenMP, CUDA). It consists of nine kernels: matrix multiplication, matrix addition, matrix convolution, matrix transpose, a Sobel filter, matrix transpose as well as Kepler's equation and ClohessyWiltshire equations.

Moreover, the SHREC GKSuite [40] exists which contains space applications, similar to ESA's OBPMark [39]. It contains 7 image processing applications: color search, histogram equalizer, image difference calculator, Mandelbrot set fractal generator, Sobel filter, bilinear thumbnailer and image tiler.

Lovelly et al. [41] benchmarked the GR740, RAD5545 and Boeing's HPSC as well as other space processing technologies and space-grade FPGAs using SHREC SpaceBench, including OpenMP parallelised versions. Later, Gretok et al. evaluated the performance of RAD5545 and Boeing's HPSC [42]. Since the actual space processors were under development at that time, their performance was estimated using COTS processors with similar characteristics.

Cannizzaro and George [43] used SHREC SpaceBench for the evaluation of a RISC-V processor under irradiation and compared the performance to the ARM cores of the Xilinx Zynq 7020, which is the predecessor of Zynq Ultrascale+. Cannizzaro [40] evaluated the same platforms also without irradiation using OpenMP and NEON versions. Belloch et al. [44] evaluated the performance of the Zynq Ultrascale+ using matrix multiplication. Landauer and Lovelly evaluated the CPU and GPU performance of a radiation tolerant NVIDIA TK1 [45] using SHREC Space Bench. Towfic et al. pre-

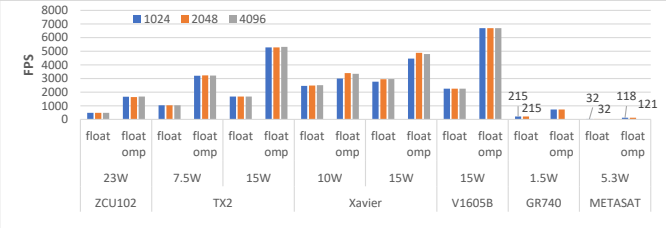


Fig. 13: CIFAR-10 Inference

sented results of benchmarking at the ISS Qualcomm's Snapdragon [46] used in Mars Ingenuity Helicopter. A set of full applications developed at JPL as well as application kernels were used, implementing NEON and OpenCL parallelisation.

Jover et al. evaluated the multicore and GPU performance of NVIDIA Xavier and AMD V1605B for CCSDS 121 and 122 compression [47], using a preliminary version of the benchmarks which are now part of ESA's OBPMark open source benchmarking suite [39]. Rodriguez evaluated NVIDIA TX2, Xavier, HiKey and AMD V1605 [48] using a port of ESA's Euclid NIR software to these GPU platforms. Rodriguez et al. [12] evaluated the multicore OpenMP and GPU performance of a superset of the platforms we cover in this work such as NVIDIA TX2, Xavier and Orin, AMD V1605B and Zynq Ultrascale+ ZCU102 and NXP's LS1046A and LX2160A, using OBPMark's image processing benchmark.

A major difference between these previous benchmarking studies and ours is that we are executing parallel OpenMP benchmarks under RTEMS-SMP instead of Linux. This makes our evaluation more relevant to the aerospace domain. Moreover, our results experience less noise from the Linux operating system, and ensure homogeneous parallel executions without pre-emptions, as the one we describe in our proposal. Another difference between the works which use the SHREC Space Bench, SHREC GKSuite and JPL software and ours is that our work relies on the equivalent GPU4S Bench and OBPMark benchmarking suites, which are open source, and allow reproducibility of the results, as well as fair comparisons to be performed among different aerospace platforms.

VI. Conclusions

In this paper we discussed the concept of the homogeneous parallelism and its benefits in the aerospace domain, both for performance and certification. We argue that this paradigm, in which all the available cores in a multicore execute the same code in a synchronous manner, simplifies multicore analysis, since the effects of contention are the same for all. We discussed implementation aspects of this computation model both for the ARINC-653 RTOS as well as for the space-qualified RTEMS RTOS, including a special case of homogeneous parallelism, vector instructions. Finally, we evaluated several multicore platforms which are considered candidates for next generation flight computers, including the space-qualified GR740, using the GPU4S Benchmarking suite parallelised with OpenMP, under RTEMS.

Our results indicate that the homogeneous parallelism can provide significant speed-ups over single core execution, with almost linear scaling. This is not only because contention is restricted to tasks executing the same code, but also due to the fact that these tasks share the last level cache and reuse the data fetched by other tasks. Regarding the performance details, the boards with 15W power mode, NVIDIA Xavier, NVIDIA TX2 and AMD V1605B provided the highest multicore performance. On the lower end of the spectrum, ZCU102 and GR740 provide similar performance, with ZCU102 being 2-3$\times$ faster, but using much higher frequency, higher power consumption and a smaller lithography node, not purposely designed for space applications. However, GR740 is the only multicore from the benchmarked ones that is qualified for space use and its harsh environment, even in institutional missions, while the rest can be used only in the avionics sector and Low Earth Orbit (LEO) for New Space Missions.

Finally, a special mention is worth on the early prototype of the METASAT platform which is currently under development on an FPGA. Based on Gaisler's NOEL-V, it provides an early preview of what can be expected by GR740's successor. Enhanced with SPARROW, a SIMD accelerator unit for AI, it provides significant speed-ups, similar to ARM's NEON. Considering also its lower frequency and the use of the lower performance GPL components of the GRLIB IP library compared to the high performance ones used in the commercial offerings of GR740 and its successor, it is expected to provide significant performance in an ASIC implementation.

Acknowledgments

The authors thank Frontgrade Gaisler for providing access to a GR740 model, upon which the relevant results were collected. This work was supported by the European Union's Horizon Europe programme under the METASAT project (grant agreement 101082622). In addition, it was supported by ESA through the 4000136514/21/NL/GLC/my co-funded PhD activity "Mixed Software/Hardware-based Fault-tolerance Techniques for Complex COTS System-on-Chip in Radiation Environments" and the GPU4S (GPU for Space) ESA-funded project. It was also partially supported by the Spanish Ministry of Economy and Competitiveness under grants PID2019-107255GB-C21 and IJC-2020-045931-I (Spanish State Research Agency / Agencia Española de Investigación (AEI) / http://dx.doi.org/10.13039/501100011033) and by the Department of Research and Universities of the Government of Catalonia with a grant to the CAOS Research Group (Code: 2021 SGR 00637).

References

[1] M. Fernández, R. Gioiosa, E. Quiñones, L. Fossati, M. Zulianello, and F. J. Cazorla, "Assessing the suitability of the NGMP multi-core processor in the space domain," in *International Conference on Embedded Software (EMSOFT)*, 2012.

[2] H. Kim and R. R. Rajkumar, "Predictable shared cache management for multi-core real-time virtualization," *ACM Trans. Embed. Comput. Syst.*, vol. 17, no. 1, Dec 2017.

[3] EASA, *AMC 20-193 Use of multi-core processors (MCPs)*, 2022.

[4] Certification Authorities Software Team (CAST), "Multi-core Processors," November 2016.

[5] L. Mutuel et al., "Limitations of interference analyses on multicore processors," in *Digital Avionics Systems Conference (DASC)*, 2017.

[6] X. Jean et al, "Assurance of Multicore Processors: Limits on Interference Analysis," FAA Final report, Tech. Rep. DOT/FAA/TC-19/24, 2020.

[7] I. Rodriguez, L. Kosmidis, J. Lachaize, O. Notebaert, and D. Steenari, "GPU4S Bench: Design and Implementation of an Open GPU Benchmarking Suite for Space On-board Processing," Universitat Politècnica de Catalunya, Tech. Rep. UPC-DAC-RR-CAP-2019-1, https://www.ac.upc.edu/app/research-reports/public/html/research_center_index-CAP-2019,en.html.

[8] J. Pérez-Cerrolaza, R. Obermaisser, J. Abella, F. J. Cazorla, K. Grüttner, I. Agirre, H. Ahmadian, and I. Allende, "Multi-core devices for safety-critical systems: A survey," *ACM Comput. Surv.*, vol. 53, no. 4, 2021.

[9] CAES, "Final Report - GR740 Next Generation Microprocessor Flight Models (NGMP Phase 3)," ESA, Tech. Rep. NGMPFM-FR-1, 2021, http://microelectronics.esa.int/finalreport/D3.18-NGMPFM-FR-1-1-0_NGMP_Final_Report.pdf.

[10] J.-L. Poupat, "DAHLIA: Very High Performance Microprocessor for Space Applications," in *ESA Workshop on Avionics, Data, Control and Software Systems (ADCSS)*, 2017.

[11] O. Notebaert et al, "NG-Ultra validation and on-board processing board development," in *Data Systems In Aerospace (DASIA)*, 2021.

[12] I. Rodríguez-Ferrández et al, "Evaluating the Computational Capabilities of Embedded Multicore and GPU Platforms for On-Board Image Processing," in *European Data Handling and Data Processing Conference for Space (EDHPC)*, 2023.

[13] L. Kosmidis, A. J. Calderón, A. Álvarez Suárez, S. Sinisi, E. Göhler, P. Gómez Molinero, A. Hönle, A. Jover Álvarez, L. Lazzara, M. Masmano Tello, P. Onaindia, T. Poggi, I. Rodríguez Ferrández, M. Solé Bonet, G. Stazi, M. M. Trompouki, A. Ulisse, V. Di Valerio, J. Wolf, and I. Yarza, "METASAT: Modular Model-Based Design and Testing for Applications in Satellites," in *Embedded Computer Systems: Architectures, Modeling, and Simulation - 22nd International Conference (SAMOS)*, ser. Lecture Notes in Computer Science, 2023.

[14] R. Berger et al., "Quad-core radiation-hardened system-on-chip power architecture processor," in *2015 IEEE Aerospace Conference*, 2015.

[15] D. Saridakis et al., "RAD55xx Platform SoC," in *Flight Software Workshop (FSW)*, 2016, https://flightsoftware.jhuapl.edu/files/2016/Day-2/Day-2-13-Saridakis.pdf.

[16] W. Powell, "High-Performance Spaceflight Computing (HPSC) Project Overview," in *Radiation Hardened Electronics Technology (RHET) Conference*, 2018.

[17] ——, "NASA's Vision for Spaceflight Computing," in *ESA Workshop on Avionics, Data, Control and Software Systems (ADCSS)*, 2022.

[18] L. Kosmidis, I. Rodriguez, A. Jover-Alvarez, S. Alcaide, J. Lachaize, O. Notebaert, A. Certain, and D. Steenari, "GPU4S: Major Project Outcomes, Lessons Learnt and Way Forward," in *Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2021.

[19] A. Pawlitzki and F. Steinmetz, "multiMIND - high performance processing system for robust NewSpace payloads," in *2nd European Workshop on On-Board Data Processing (OBDP2021)*, 2021, https://doi.org/10.5281/zenodo.5521502.

[20] L. Kosmidis, C. Maxim, V. Jégu, F. Vatrinet, and F. J. Cazorla, "Industrial Experiences with Resource Management Under Software Randomization in ARINC653 Avionics Environments," in *International Conference on Computer-Aided Design (ICCAD)*, 2018.

[21] A. Agrawal et al., "Contention-Aware Dynamic Memory Bandwidth Isolation with Predictability in COTS Multicores: An Avionics Case Study," in *Euromicro Conference on Real-Time Systems, (ECRTS)*, 2017.

[22] R. Pujol et al., "Empirical Evidence for MPSoCs in Critical Systems: The Case of NXP's T2080 Cache Coherence," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021.

[23] N. Sensfelder et al., "On How to Identify Cache Coherence: Case of the NXP QorIQ T4240," in *Euromicro Conference on Real-Time Systems (ECRTS)*, 2020.

[24] D. Radack et al., "Civil certification of multi-core processing systems in commercial avionics," in *Safety-critical Systems Symposium*, 2018.

[25] S. H. VanderLeest and D. C. Matthews, "Incremental Assurance of Multicore Integrated Modular Avionics (IMA)," in *Digital Avionics Systems Conference (DASC)*, 2021.

[26] ARINC, *ARINC Specification 653: Avionics Application Software Standard Interface*, Aeronautical Radio, Inc, 2013.

[27] P. Gómez Molinero et al., "De-RISC: Dependable Real-time RISC-V Infrastructure for Safety-critical Space and Avionics Computer Systems," in *Data Systems In Aerospace (DASIA)*, 2021. [Online]. Available: https://doi.org/10.5281/zenodo.5707537

[28] *RTEMS SMP QDP*. ESA, 2022. [Online]. Available: https://rtems-qual.io.esa.int

[29] K. Hoyme and K. Driscoll, "SAFEbus," in *11th Digital Avionics Systems Conference (DASC)*, 1992.

[30] A. Melani et al., "A scheduling framework for handling integrated modular avionic systems on multicore platforms," in *International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2017.

[31] P. Bretault et al., "Approaching Parallelization of Payload Software Application on ARM Multicore Platforms," in *Data Systems in Aerospace (DASIA)*, 2015.

[32] RTEMS, *OpenMP*, RTEMS, 2015.

[33] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[34] D. Steenari et al., "On-Board Processing Benchmarks," 2021, http://obpmark.github.io/.

[35] M. Solé and L. Kosmidis, "Compiler Support for an AI-Oriented SIMD Extension of a Space Processor," *Ada Lett.*, vol. 42, no. 1, 2022.

[36] M. S. Bonet and L. Kosmidis, "SPARROW: A Low-Cost Hardware/Software Co-designed SIMD Microarchitecture for AI Operations in Space Processors," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022.

[37] L. Kosmidis, M. Solé Bonet, I. Rodriguez-Ferrández, J. Wolf, and M. M. Trompouki, "The METASAT Hardware Platform: A High-Performance Multicore, AI SIMD and GPU RISC-V Platform for On-board Processing," in *European Data Handling and Data Processing Conference for Space (EDHPC)*, 2023.

[38] R. Roger et al., "Vector extensions in COTS processors to increase guaranteed performance in real-time systems," *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 2, 2023.

[39] D. Steenari, L. Kosmidis, I. Rodríguez-Ferrández, A. Jover-Álvarez, and K. Förster, "OBPMark (On-Board Processing Benchmarks) - Open Source Computational Performance Benchmarks for Space Applications," in *2nd European Workshop on On-Board Data Processing (OBDP)*, 2021. [Online]. Available: https://doi.org/10.5281/zenodo.5638577

[40] M. J. Cannizzaro, "RISC-V Benchmarking for Onboard Sensor Processing," Master's thesis, University of Pittsburgh, 2019, https://d-scholarship.pitt.edu/40400/1/cannizzaromichaeljames_etdPitt2021.pdf.

[41] T. M. Lovelly et al., "Benchmarking Analysis of Space-Grade Central Processing Units and Field-Programmable Gate Arrays," *Journal of Aerospace Information Systems*, vol. 15, no. 8, 2018.

[42] E. W. Gretok et al., "Comparative Benchmarking Analysis of Next-Generation Space Processors," in *IEEE Aerospace Conference*, 2019.

[43] M. J. Cannizzaro and A. D. George, "Evaluation of RISC-V Silicon Under Neutron Radiation," in *2023 IEEE Aerospace Conference*, 2023.

[44] J. A. Belloch et al., "Evaluating the computational performance of the Xilinx Ultrascale+ EG Heterogeneous MPSoC," *J. Supercomput.*, vol. 77, no. 2, 2021.

[45] D. C. Landauer and T. M. Lovelly, "Performance Evaluation of the Radiation-Tolerant NVIDIA Tegra K1 System-on-Chip," in *2023 IEEE Space Computing Conference (SCC)*, 2023.

[46] Z. Towfic et al., "Benchmarking and Testing of Qualcomm Snapdragon System-on-Chip for JPL Space Applications and Missions," in *IEEE Aerospace Conference (AeroConf)*, 2022.

[47] A. Jover-Alvarez, I. Rodríguez, L. Kosmidis, and D. Steenari, "Space Compression Algorithms Acceleration on Embedded Multi-Core and GPU Platforms," *Ada Lett.*, vol. 42, no. 1, dec 2022.

[48] I. Rodríguez Ferrández, "An On-board Algorithm Implementation on an Embedded GPU: A Space Case Study," Master's thesis, Universitat Politcnica de Catalunya, 2021, https://upcommons.upc.edu/handle/2117/344892.